Experiment No. 4

Title:

Apply memoization and tabulation techniques to efficiently compute the nth Fibonacci number using dynamic programming.

Aim:

The aim of this program is to efficiently compute the nth Fibonacci number using techniques that optimize time complexity, ensuring that the computation is fast and scalable even for large values of n.

Theory:

Understanding the Fibonacci Sequence

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, starting from 0 and 1. In mathematical terms, the sequence is defined recursively as:

- F(0) = 0
- F(1) = 1
- F(n) = F(n-1) + F(n-2) for n > 1

Efficient Approaches

1. **Iterative Approach**:
    - This approach iterates from the bottom up, calculating each Fibonacci number sequentially until it reaches the desired nth Fibonacci number.
    - Time complexity: O(n)
    - Space complexity: O(1)
2. **Recursive Approach:**
    - *In mathematical terms, the sequence Fn of Fibonacci numbers is defined by the recurrence relation: $Fn=Fn−1+Fn−2$ $Fn =Fn−1 +Fn−2$ with seed values and $F0=0$ $F0 =0$ and $F1=1$ $F1 =1$.*
    - The Nth Fibonacci Number can be found using the recurrence relation shown above:
        - if $n = 0$, then return 0.
        - If n = 1, then it should return 1.
        - For n > 1, it should return $F_{n-1} + F_{n-2}$

Pseudocode:

```
def fibonacci_iterative(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    a, b = 0, 1
```

```
    for _ in range(2, n + 1):
        c = a + b
        a = b
        b = c
    return b
```

Experiment 4 Title:

Experiment 4 Code:

Experiment 4 Output (Screenshot):

Bucket List Experiment Title:

Bucket List Experiment Code:

Bucket List Experiment Output (Screenshot):

Theory Questions (Handwritten):

Q1: Explain the difference between memoization and tabulation in dynamic programming. When would you choose one method over the other?

Q2: What is meant by "overlapping subproblems" in dynamic programming, and why is this concept important for solving optimization problems efficiently?

Conclusion:

Thus program calculates the nth Fibonacci number efficiently by using an iterative approach with dynamic programming, significantly reducing both time and space complexity compared to naive recursive methods.